

Senior Engineer Product Document

Openbentt

Local-first, privacy-respecting AI chat client — Web + Electron Desktop

Version 2.0.6

Date May 2026

Scope Full product — web & desktop

Status Active development

CONTENTS

1. Executive Summary
2. Problem Statement
3. What Openbentt Does
4. System Architecture
5. Feature Inventory
6. Current State Assessment
7. UX Improvement Plan
8. Testing Plan
9. Phased Completion Roadmap
10. Distribution & Release Plan
11. Tech Debt & Known Issues
12. Success Metrics

1. Executive Summary

Openbentt is a **local-first, privacy-respecting multi-provider AI chat client** packaged as both a static web app and an Electron desktop application. Users bring their own API keys (OpenRouter, OpenAI, Anthropic, Google Gemini, or any OpenAI-compatible endpoint) and run chats, document workspaces, research pipelines, and on-device language models — all without any backend account system.

Unique Value	Why it matters
API keys never leave the user's browser or device	No server-side account, no data leakage risk
On-device inference (WebGPU/WASM + native GGUF)	Works offline; privacy-critical use cases
Compare 2–4 models side-by-side with real timing	Inform model selection decisions with actual data
Document-grade workspaces (LaTeX ↔ AI ↔ PDF)	Academic writing + AI in one surface
Research augmentation with citation graphs	BibTeX, arXiv, Semantic Scholar, HF datasets

Current maturity

The core chat engine and workspace features are functionally complete. Key gaps: three workspace routes are hidden from nav, no integration/E2E tests, llama-server not bundled in releases, and macOS/Windows builds are unsigned.

2. Problem Statement

User Pain	Current tools fail because...	Openbentt's answer
API keys sent to third-party chat wrappers	Keys stored server-side; breach risk	Keys in browser localStorage; zero server-side account
Can't compare models on my own prompts	One model per tab; manual copy-paste	Parallel 2–4 model comparison, one send, live metrics
Cloud-only inference — privacy concerns	Every token goes to a cloud provider	On-device Gemma/Qwen (WebGPU/CPU) + GGUF via llama-server
Academic writing tools are separate from AI	Overleaf + ChatGPT in separate tabs	Notebook: AI ↔ LaTeX/PDF ↔ diff/apply in one surface
Gated HF model tokens scattered	Manual env vars on every machine	Electron: encrypted HF token via OS keychain (safeStorage)
No way to share a conversation	Screenshot or copy-paste	Share URL via compressed hash snapshot (no server needed)

3. What Openbentt Does — In Plain Language

3.1 Core Chat

Pick a provider and model, type a message, get a streamed reply. All threads stored in browser localStorage — no account, no server. Multiple threads with auto-named titles.

3.2 Model Comparison

Send one prompt → parallel replies from 2–4 models. Each reply shows **Time to First Token**, total response time, and token counts. Great for model evaluation on real tasks.

3.3 On-Device Inference

- **WebGPU / WASM (web + desktop):** `@huggingface/transformers` runs Gemma, Qwen-0.5B, etc. inside the browser. Auto-falls back: WebGPU → WASM CPU → smaller model.
- **Local GGUF (desktop only):** Download any GGUF from Hugging Face in the Labs panel. Electron spawns `llama-server` on localhost; the chat stack talks to it as an OpenAI-compatible API.

3.4 Notebook Workspace

Upload a PDF or write LaTeX. Ask AI to improve sections, add citations, fix errors. Changes diff before applying. Compiles PDF via BusyTeX WASM (in-browser) or remote `pdflatex` server.

3.5 Research Labs

- BibTeX management and citation graph visualization
- Hugging Face dataset card browser
- Local GGUF Hub: search, download, manage, and chat with models
- Research proxy: enriches queries with Wikipedia, arXiv, Semantic Scholar, Brave Search

3.6 Additional Workspaces

- **LaTeX Write (`/write`):** KaTeX preview + PDF export — lighter than full Notebook
- **Benchmark (`/benchmark`):** Run a prompt N times across models; plot latency distributions
- **WebGPU Diagnostics (`/webgpu`):** Probe GPU, check adapter, run Gemma smoke test

3.7 Share, Export, Settings

Share any chat as a compressed URL (no server). Export to Markdown or PDF. Full settings: provider, model, theme, research toggle, GGUF path, HF token, keyboard shortcuts.

4. System Architecture

4.1 Three-Layer Model

Electron main process (Node) [desktop only]

app:// protocol → dist/ · BrowserWindow · IPC handlers · spawns llama-server (127.0.0.1) · HF download · userData registry · safeStorage HF token

Preload bridge (contextBridge)

Exposes `window.openbenttLocalGguf` and `window.openbenttDesktop` — no Node in renderer, context isolation, sandbox

Renderer — React SPA (same for web + desktop)

ChatContext → routes by `aiProvider` → OpenRouter (HTTPS) / WebGPU WASM (in-process) / Local GGUF (HTTP loopback via IPC) · localStorage keys + chat history

4.2 AI Provider Routing

```
ChatContext.runAssistantPipeline
├─ "openrouter"      → streamOpenRouterChat (HTTPS to api.openrouter.ai)
├─ "openai_direct"   → streamOpenRouterChat w/ openai base URL
├─ "openai_compatible" → streamOpenRouterChat w/ custom base URL (Ollama, LM Studio...)
├─ "webgpu_gemma"    → streamLocalGemmaChat (Transformers.js in-renderer)
│   └─ ensureLocalGemmaLoaded → WebGPU preferred, WASM fallback, tiny-model fallback
└─ "local_gguf"      → streamLocalGgufChat
    └─ IPC: ensureServer(registryId) → main spawns llama-server
        └─ HTTP POST 127.0.0.1:<port>/v1/chat/completions (OpenAI-compatible stream)
```

4.3 Key Files Reference

Area	Files
App entry + routing	src/App.tsx , src/main.tsx
Core state machine	src/context/ChatContext.tsx (~900 lines)
Domain types	src/types/chat.ts
AI streaming	src/lib/aiStream.ts , openrouter.ts
WebGPU / WASM local	src/lib/gemmaWebGpu/*
GGUF / llama-server	src/lib/localGguf/*
Notebook + LaTeX	src/lib/compileNotebook.ts , latexWasmCompile.ts , 8+ latex helpers
Research	src/lib/researchSources.ts , bibtex.ts , citationGraph.ts
Electron shell	electron/main.mjs , localGgufService.mjs , hfSecretStore.mjs
Electron bridge	electron/preload.cjs
Optional servers	server/research-proxy.mjs , server/latex-compile.mjs

4.4 Deploy Modes

Mode	What runs	Features available
Static hosting	Just dist/ on CDN/nginx	Chat, comparison, WebGPU, share, notebook (WASM LaTeX)
Docker	nginx + research-proxy.mjs	All static + research proxy enrichment (+ Brave if key set)
Docker + LaTeX	nginx + research + latex-compile.mjs	All above + server-side pdflatex (for full document compile)
Electron desktop	Chromium + main.mjs	All above + GGUF + safeStorage HF token + auto GPU flags

5. Feature Inventory

5.1 Complete Done

Feature	Evidence
Multi-provider chat streaming	<code>aiStream.ts</code> , <code>ChatContext</code>
OpenRouter model list + free-tier highlights	<code>openrouter.ts</code> , <code>useOpenRouterModels.ts</code>
Multi-thread chat (titles, search, history)	<code>ChatContext</code> , <code>ChatThreadBar</code>
Parallel model comparison (2–4)	Comparison pipeline in <code>ChatContext</code>
Multimodal attachments (image, audio, video, PDF)	<code>ChatInput</code> , <code>ChatContext</code>
In-chat charts (<code>openbentt-chart</code> fences)	<code>chartSpec.ts</code> , <code>OpenbenttChartViews</code>
Share URL — hash snapshot (no server)	<code>shareRun.ts</code> , <code>ShareViewPage</code>
Export: Markdown + PDF	<code>chatExportMarkdown.ts</code> , <code>chatExportPdf.ts</code>
Keyboard shortcuts	<code>KeyboardShortcutsSheet</code>
Prompt snippets (saved, local)	<code>promptSnippets.ts</code>
Context window meter + quota meter	<code>contextMeter.ts</code> , <code>ProviderQuotaMeter</code>
Dark / light theme	<code>ThemeContext</code>
WebGPU / WASM local Gemma	<code>gemmaWebGpu/*</code>
Local GGUF download + llama-server lifecycle	<code>localGgufService.mjs</code> , <code>LocalGgufHub</code>
HF token encrypted storage (desktop)	<code>hfSecretStore.mjs</code> (<code>safeStorage</code>)
Notebook LaTeX/PDF workspace	<code>NotebookPdfWorkspace</code> , <code>compileNotebook</code>
Research Labs (BibTeX, citation graph, HF datasets)	<code>ResearchLabsPage</code> , <code>bibtex.ts</code> , <code>citationGraph.ts</code>
Research proxy server (Docker)	<code>server/research-proxy.mjs</code>
LaTeX compile server	<code>server/latex-compile.mjs</code>
Docker deploy (nginx + proxy)	<code>Dockerfile</code> , <code>docker-compose.yml</code>
Electron packaging (AppImage, deb, NSIS, dmg)	electron-builder config
CI: lint + test + build	<code>.github/workflows/ci.yml</code>
Release pipeline: tag → multi-platform GitHub Release	<code>.github/workflows/release.yml</code>
Storage migration (legacy cogerphere-* keys)	<code>storageMigrate.ts</code>

5.2 Partial / Needs Work Partial

Feature	What exists	What's missing
Route discovery for /write, /benchmark, /webgpu	Routes exist and work	Not in sidebar nav — users can't find them
WebGPU setup messaging	SetupPage blocks if <code>!navigator.gpu</code>	WASM/CPU always works; messaging should reflect this
llama-server distribution	Code handles bundled binary path	Binary not in CI artifacts; users hit "not found"
Comparison mode disabled for local providers	Correctly disabled	No UI explanation of why
LaTeX Write workspace	<code>LatexWorkspacePage.tsx</code> renders	No sidebar link; PDF export is raster not vector
Benchmark page	Works	No sidebar link; results not persisted; no export
Mobile layout	Responsive classes present	Keyboard overlap; not touch-optimized
First-run onboarding	SetupPage for API key	No guide for desktop GGUF; no feature discovery tour

5.3 Not Started ✖ Missing

Feature	Priority	Notes
Electron auto-update	High	electron-updater not configured; users manually re-download
Code signing / notarization	High	Unsigned builds trigger Gatekeeper/SmartScreen
E2E / integration tests	High	Zero coverage for chat flow, routing, Electron IPC
Component tests (React)	Medium	No <code>.test.tsx</code> files; ChatContext entirely untested
System prompt editor (per-thread)	Medium	Only workspace-injected; no user-editable prompt
True inline user message edit	Medium	Current: reloads composer. Missing: inline edit + auto-regen
Conversation branching	Medium	No fork from message; retry replaces in-place
Model pinning / favorites	Low-Med	Model selector has no pin
Historical token / cost dashboard	Low-Med	Per-session metrics only
Structured tool-call display	Low-Med	Tool calls appear as raw text
Voice input / TTS	Low	Not present
Cross-device sync	Low	localStorage only

6. Current State Assessment

What works well

- Core chat loop is solid; unit-tested parsers and helpers
- Provider abstraction in `aiStream.ts` is clean and extensible
- Notebook workflow is genuinely powerful (LaTeX ↔ AI ↔ PDF in one surface)
- Electron shell is well-secured (context isolation, sandbox, no Node in renderer)
- GGUF download / llama-server lifecycle is robust (resume download, disk check, SHA-256, cleanup on quit)
- HF token handling uses OS-level encryption on desktop

Core concerns

Discoverability gap

Three full workspaces (`/write` , `/benchmark` , `/webgpu`) are invisible to users navigating via the sidebar. The product is larger than it appears from the nav.

Testing gap

`ChatContext.tsx` is the application's core state machine (~900 lines) and has zero test coverage. A provider or streaming change could silently break core chat.

Distribution gap

`llama-server` is the centerpiece of the desktop's local AI story but is not bundled in CI artifacts. Most desktop users hit a manual installation step.

Signing gap

macOS and Windows builds are unsigned. This is a blocking issue for non-technical users who see Gatekeeper / SmartScreen warnings.

Documentation drift

`PRODUCTION_CHECKLIST.md` and `electron/README.md` have stale references (`preload.mjs` vs actual `preload.cjs` ; Electron first-load path note).

7. UX Improvement Plan

7.1 Navigation & Discovery

- Add `/write`, `/benchmark`, `/webgpu` to `WORKSPACE_NAV_ITEMS` with appropriate icons (PenLine, BarChart2, Cpu)
- Add a collapsible "Advanced" group in the sidebar if it becomes crowded
- Add "What's new / Download latest" callout to Settings when a newer GitHub Release is detected

7.2 Setup & Onboarding

- Rewrite the on-device setup card: remove hard GPU block; say "GPU recommended, CPU also works"
- Show a hardware detection badge with the actual result (WebGPU / WASM / GPU model)
- Desktop first-run: 3-step guided modal for GGUF ("Download a model → Pick Local GGUF → Chat"), gated by `userData/onboarded.json`

7.3 Chat Interface

- True inline user message edit with confirmation + auto-regenerate (replaces composer-reload)
- Retry history: keep previous responses behind "See previous" toggle (like ChatGPT retries)
- Detect OpenAI tool-call JSON in assistant replies → render as collapsible "Tool call" cards
- System prompt editor per thread (text field in thread settings sidebar)

7.4 Mobile

- Use `visualViewport` resize handler to push chat input above keyboard
- Swipe-to-switch-thread gesture on mobile viewport
- Set `overflow: hidden` on body when soft keyboard is open to prevent scroll jank

7.5 Settings Refactor

- Split single large panel into tabbed sections: General · Models · Providers · Desktop (Electron-only) · Privacy · Advanced
- Desktop tab only visible when `window.openbenttDesktop?.isElectron` is true

7.6 Error States

- Parse HTTP 429 `x-ratelimit-reset-requests` header → show "Try again in X sec" countdown
- GGUF "binary not found" → show diagnostic checklist UI:  Model downloaded ·  llama-server missing → [Install guide]
- WebGPU load progress: show download % + estimated time for model weights
- On API key missing: show actionable empty state with button → Settings (not just an error toast)

7.7 Performance

- Add proper loading skeleton to `LocalGgufHub` while registry loads
- Use `startTransition` for model selector updates to avoid blocking input
- Verify Rollup `manualChunks` splits are actually reducing initial bundle size (measure with `vite-bundle-analyzer`)
- Lazy-load heavy workspace deps (busytex, pdfjs, transformers) — partially done; validate with Lighthouse

8. Testing Plan

8.1 Current Coverage Map

Layer	Current	Target
Lib unit tests (parsers, helpers)	~22 files, solid	Keep + expand (+10 files)
ChatContext / state machine	0	Full coverage of critical paths
Component tests (.test.tsx)	0	ChatInput, SettingsPanel, LocalGgufHub
Route / integration tests	0	8 happy-path flows with MSW
Electron main-process unit tests	0	IPC handlers, binary resolution, disk checks
E2E (Playwright)	0	7 user-journey scenarios
Server tests (research-proxy, latex-compile)	0	Basic request/response coverage

8.2 Unit Tests to Add (Immediate)

ChatContext (`src/context/ChatContext.tsx`)

- `canSendChat` returns correct boolean for each provider + config combination
- `normalizeApiConfig` : migration of deprecated defaults; model reset on provider switch
- Message append + streaming accumulation (mock stream)
- Error handling: network error, AbortSignal, HTTP 429 rate limit
- Comparison mode: parallel requests started, dedup applied, all replies arrive

Components

- `ChatInput` : submit on Enter, Shift+Enter newline, attachment add/remove, snippet insert
- `SettingsPanel` : switching provider resets model field to provider default
- `LocalGgufHub` : registry empty state, download in-progress state, "Use in chat" button

Electron main-process (`electron/localGgufService.mjs`)

- `resolveLlamaServerBinary` : env > settings > bundled > PATH precedence
- `getDiskFreeBytesApprox` : parse `df` output correctly
- IPC payload validation: invalid `repoId` throws; non-`.gguf` filename throws

8.3 Integration Tests (Vitest + @testing-library/react + MSW)

- Full chat send → receive → display: mock OpenRouter response with streaming
- Share flow: encode thread → decode → render read-only view
- Notebook: upload PDF text → AI apply suggestion → diff view → accept
- Research: proxy returns sources → context injected in next message payload
- Setup page: incomplete config → redirect to `/setup` ; complete config → `/chat`
- Comparison: two models → both response streams complete → metrics displayed

8.4 E2E Tests (Playwright)

Scenario	Rationale
New user: enter API key → send message → receive reply	Core loop regression
Compare mode: pick 2 models → send → both reply with metrics	Comparison regression
Export chat as Markdown: download file contains messages	Data portability
Share: encode → open share URL → read-only view renders	Sharing
Settings: toggle dark mode → persists after page reload	Persistence
Setup page: enter valid key → redirects to /chat	Onboarding
Desktop: download GGUF → select local GGUF provider → send message	Desktop GGUF regression

8.5 CI Integration

- **Every PR:** unit tests + component tests + integration tests (`npm run test`)
- **Main push + nightly:** E2E suite (Playwright, headless Chromium)
- **Release tag builds:** Electron E2E after packaging, macOS notarization check

9. Phased Completion Roadmap

0 Foundation Cleanup

1-2 weeks

Goal: Remove confusion, fix docs drift, make hidden routes discoverable.

- Delete unused `src/lib/agentWorkflow.ts` (no imports)
- Delete or repurpose unused `src/components/PlaygroundShell.tsx`
- Fix `electron/README.md`: `preload.mjs` → `preload.cjs`
- Fix `PRODUCTION_CHECKLIST.md`: stale Electron first-load path entry
- Add `/write`, `/benchmark`, `/webgpu` to `WORKSPACE_NAV_ITEMS` with icons
- Write 5 highest-value unit tests for `ChatContext` (`canSendChat`, `normalizeApiConfig`)
- Update `CHANGELOG.md` with all [Unreleased] changes
- Sync Download page links to current artifact URLs

1 Chat Polish

2-3 weeks

Goal: Core chat loop is delightful and production-ready.

- True inline user message edit (replace composer-reload behavior)
- Retry history: keep previous assistant responses; "See previous" toggle
- Mobile: `visualViewport` keyboard push-up fix
- Settings: split into tabbed sections (General, Models, Providers, Desktop, Privacy, Advanced)
- Error UX: rate-limit countdown timer; GGUF diagnostic checklist
- Actionable empty states when API key missing / no model selected
- Model selector: favorites / pin to top
- Thread sidebar: collapse/expand by date groups
- Add `startTransition` for model switch
- Component tests: `ChatInput`, `SettingsPanel`

2 Local Models & Desktop

2-3 weeks

Goal: Local inference works out-of-the-box; desktop is a first-class experience.

- Bundle `llama-server` per-platform in CI via `electron-builder` `extraResources`
- macOS code signing + notarization secrets in GitHub Actions
- Desktop first-run guide (3-step onboarding modal gated by `userData/onboarded.json`)
- Fix Setup page: WASM/CPU path always available; remove hard GPU block
- UI explanation when comparison mode is disabled for local providers
- GGUF: download progress with ETA + speed; pause/resume button
- GGUF: show VRAM estimate + parameter count before download
- WebGPU: show active backend (WebGPU / WASM / CPU) in thread header
- Electron auto-update via `electron-updater` (check on startup)
- Single-instance lock (`app.requestSingleInstanceLock()`)
- Electron main-process unit tests + Playwright Electron E2E

3 Workspaces & Research

2-3 weeks

Goal: Notebook, Research Labs, Write, and Benchmark are polished and integrated.

- Notebook: progress indicator during WASM PDF compilation
- Notebook: auto-save draft to `localStorage` (prevent data loss on reload)
- Notebook: side-by-side diff view before apply
- Research Labs: attach research context directly to a chat thread
- GGUF Hub: add VRAM badge + parameter count from HF model card
- `/write` workspace: add to sidebar nav; upgrade PDF export to vector
- Benchmark: persist results to `localStorage`; export as CSV; add to sidebar nav
- Agent workflow: implement basic multi-step (search → summarize → cite)
- Per-thread system prompt editor
- Integration tests: Notebook apply-from-chat; research proxy flow

4 Quality & Hardening

2 weeks

Goal: Production-ready with confidence; security and accessibility solid.

- Full E2E suite (Playwright): all flows from Phases 1-3
- Server tests: `research-proxy.mjs`, `latex-compile.mjs`
- IPC handlers: Zod validation for all `localGguf:*` payloads
- CSP headers in Docker nginx config (currently absent)
- Security audit: no path traversal in IPC; no unintended `localStorage` exposure
- Performance profiling: cold start, first chat latency, WASM init time
- Bundle audit: remove unused dependencies (`knip` / `depcheck`)
- Accessibility: ARIA roles for messages, keyboard nav, focus management
- Error boundary: add opt-in crash reporting (no chat content)
- Finalize and sign off `PRODUCTION_CHECKLIST.md`

Goal: Users can discover, install, and update with zero friction.

- GitHub Releases: all platform artifacts correctly named + checksums attached
- Landing page: update with current feature set + download buttons
- Auto-update channel live (electron-updater → GitHub releases)
- Docker image published to GitHub Container Registry (GHCR) with version tags
- SEO: verify `VITE_PUBLIC_SITE_URL` , Open Graph tags, sitemap
- Version bump workflow documented: `npm version patch/minor/major` + tag → pipeline

10. Distribution & Release Plan

Current Pipeline



Gaps & Fixes

Gap	Fix	Priority
llama-server not in artifacts	Add <code>extraResources</code> to electron-builder; build per-platform in CI	High
macOS unsigned	Apple Developer cert + notarization in <code>build-macos</code> job (secrets: <code>APPLE_ID</code> , <code>CSC_LINK</code> ...)	High
Windows unsigned	Authenticode cert; document workflow as "coming soon" for now	Medium
No auto-update feed	<code>electron-updater</code> + <code>publish</code> config in electron-builder pointing at GitHub	High
Docker not published	Add <code>docker/build-push</code> step in release workflow → GHCR	Medium
No checksums	Add <code>sha256sum</code> step; attach <code>checksums.txt</code> to release	Medium
Download page URL drift	<code>releaseDownloads.ts</code> should read <code>package.json</code> version dynamically	Medium

Versioning Strategy

- **Patch** (2.0.x): bug fixes, doc updates, minor UX tweaks
- **Minor** (2.x.0): new features, new providers, new workspaces
- **Major** (x.0.0): breaking changes — storage schema, API contract changes

11. Tech Debt & Known Issues

Item	Severity	Location
<code>agentWorkflow.ts</code> exported but never imported	Low	<code>src/lib/agentWorkflow.ts</code>
<code>PlaygroundShell.tsx</code> appears unused	Low	<code>src/components/PlaygroundShell.tsx</code>
<code>src/pages/Index.tsx</code> dead re-export (nothing imports it)	Low	<code>src/pages/Index.tsx</code>
<code>electron/README.md</code> says <code>preload.mjs</code> (should be <code>preload.cjs</code>)	Medium	<code>electron/README.md</code>
<code>PRODUCTION_CHECKLIST.md</code> Electron path item stale	Medium	<code>PRODUCTION_CHECKLIST.md</code>
No CSP in Docker nginx config	High	<code>docker/nginx-docker.conf</code>
IPC payloads: no Zod validation (trust renderer inputs)	Medium	<code>electron/localGgufService.mjs</code>
<code>ChatContext.tsx</code> ~900 lines; zero test coverage	High	<code>src/context/ChatContext.tsx</code>
No single-instance lock in Electron	Medium	<code>electron/main.mjs</code>
Download page release URLs may drift from actual releases	Medium	<code>src/config/releaseDownloads.ts</code>
<code>use-toast.ts</code> duplicated in <code>hooks/</code> and <code>components/ui/</code>	Low	Both locations
No <code>localStorage</code> quota management; can fill up silently	Medium	<code>ChatContext</code> storage writes
WebGPU WASM cold load (30–120s) with no meaningful progress	High	<code>WebGpuPage</code> , <code>LocalOnDeviceModelBar</code>

12. Success Metrics

Metric	Current	Target (Phase 5)
Core unit test files (lib/)	~22 files	32+ files incl. ChatContext
Integration test flows	0	8 happy paths
E2E scenarios	0	7
Hidden workspace routes discoverable	0 / 3	3 / 3
Electron macOS signed + notarized	No	Yes
llama-server bundled in release	No	Yes (all platforms)
Auto-update (Electron)	No	Yes
CSP in Docker nginx	No	Yes
WebGPU load progress feedback	Minimal	% + ETA shown

Release Readiness Checklist

- ☐ `npm run lint` passes
- ☐ `npm run test` passes (all unit tests)
- ☐ E2E suite passes against latest build
- ☐ macOS build signed + notarized
- ☐ llama-server bundled for all platforms
- ☐ CHANGELOG.md updated
- ☐ Download page URLs match new release
- ☐ PRODUCTION_CHECKLIST.md reviewed and signed off
- ☐ Docker image builds and serves correctly
- ☐ Version tag applied and pipeline green